Elevator Controller Design Using Finite State Machine (FSM)

RTL Design, Simulation, and Synthesis using Verilog HDL

Internship Project Report

Domain: VLSI

Submitted By:

Shambhavi Bansode

Electronics and Telecommunication Engineering,

NIT Kurukshetra

Submitted To:

Internship Evaluation

Project:

3-Floor Elevator Controller

Table of Contents

Sr. No.	Contents	Page No.
1	Title Page	1
2	Certificate	2
3	Acknowledgement	3
4	Aim and Objectives	4
5	Theory	5
6	Design Methodology	6
7	State Diagram	7
8	Verilog HDL Code	8
9	Verilog Code Explanation	13
10	Testbench Code	12
11	Testbench Explanation	18
12	Simulation Results	19
13	RTL Schematic	20
14	Simulation Log	22
15	**Functional Verification**	**22**
16	Conclusion	27

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Skill Flow** for providing me with the opportunity to work on this internship project. This project has helped me gain practical knowledge of **Verilog HDL**, **Finite State Machine (FSM) design**, **RTL synthesis**, and **functional simulation using Xilinx Vivado**.

I am thankful to the mentors and instructors at Skill Flow for their valuable guidance, continuous support, and technical assistance throughout the project. Their insights greatly enhanced my understanding of digital design concepts and FPGA-based development.

I would also like to thank my college, teachers, and everyone who directly or indirectly contributed to the successful completion of this project.

Finally, I am grateful for this learning experience, which has strengthened my practical skills and deepened my interest in digital system design and VLSI.

Elevator Controller using Verilog HDL

Aim

To design, simulate, and verify a Finite State Machine (FSM)-based Elevator Controller for a three-floor building using Verilog HDL in Xilinx Vivado.

Objective

- Design an FSM-based elevator controller.
- Accept floor requests from users.
- Control elevator movement between floors.
- Operate door opening and closing.
- Verify the design through simulation.
- Generate the synthesized RTL schematic.

Theory

An elevator controller is a sequential digital system that manages the movement of an elevator based on floor requests. Since its operation depends on both the current state and input signals, it is implemented using a **Finite State Machine (FSM)**.

The controller continuously monitors floor requests and determines whether the elevator should move upward, move downward, remain idle, or operate the door. After reaching the requested floor, the controller opens the door, waits for a door-close command, and then returns to the idle state.

In this project, Verilog HDL is used to describe the FSM, while Xilinx Vivado is used for simulation and RTL synthesis.

Software and Hardware Requirements

Software

- Xilinx Vivado 2025.2

Hardware

- Personal Computer
- Windows Operating System

Design Methodology

The elevator controller is designed using five FSM states.

State	Description
IDLE	Waits for floor requests
MOVE_UP	Elevator moves upward
MOVE_DOWN	Elevator moves downward
DOOR_OPEN	Opens the elevator door
DOOR_CLOSE	Closes the elevator door

The controller accepts requests for Floors 1, 2, and 3. Based on the current floor and target floor, the FSM determines the movement direction. Once the destination is reached, the door opens and remains open until a close command is received.

Inputs and Outputs

INPUTS

Signal	Description
clk	System Clock
reset	Resets the controller
req_floor1	Floor 1 request
req_floor2	Floor 2 request
req_floor3	Floor 3 request
door_close	Door close signal

OUTPUTS

Signal	Description
current_floor	Current elevator floor
direction	Elevator direction
moving	Elevator movement indicator
door_open	Door status

Sr. No._____ Date:_____

* The O/P depends → entirely on present state of the elevator → ∴ Moore FSM

MOORE FSM STATE DIAGRAM

① Assumption

① Elevator starts at floor 1 after reset.

② Only 1 floor request at a time is considered.

③ Door remains open, until door close.

④ After door closes, elevator goes to Idle state.

② Inputs

① clk
② reset (active high)
③ req_floor1
④ req_floor2
⑤ req_floor3
⑥ door_close

③ Outputs

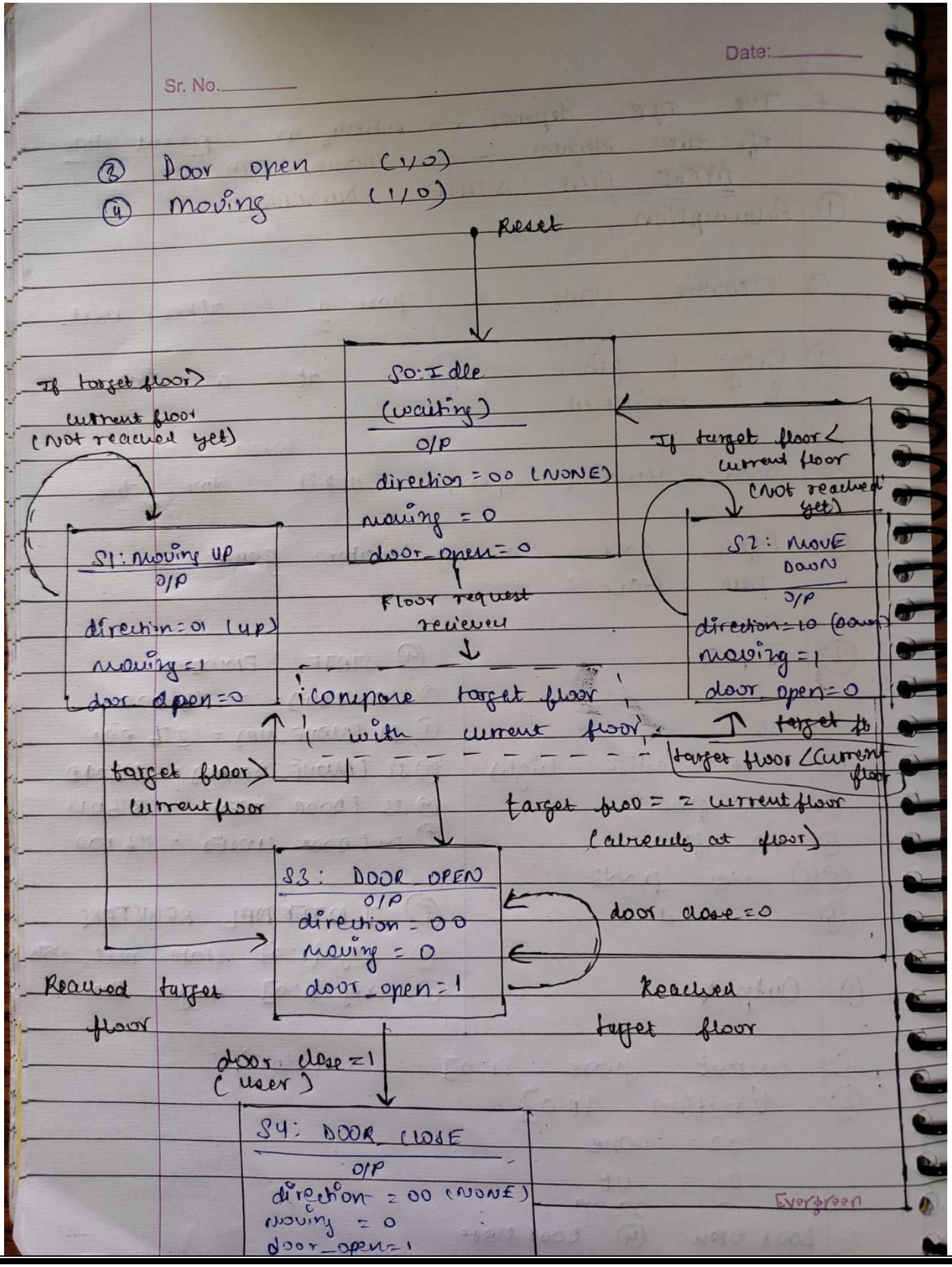
① current_floor [1:0] (1 to 3)
② direction [1:0]
00 = none
01 = UP
10 = Down
③ DOOR Open ④ DOOR close

④ STATE ENCODING
① S0 (IDLE) = 3'b000
② S1 (MOVE_UP) = 3'b001
③ S2 (MOVE_DOWN) = 3'b010
④ S3 (DOOR_OPEN) = 3'b011
⑤ S4 (DOOR_CLOSE) = 3'b100

⑤ INTERNAL REGISTERS
① reg [2:0] state, next_state
② reg [1:0] target_floor.

Evergreen

State Diagram



Verilog code

```
module elevator_controller(
    input clk,
    input reset,
    input req_floor1,
    input req_floor2,
    input req_floor3,
    input door_close,

    output reg [1:0] current_floor,
    output reg [1:0] direction,
    output reg door_open,
    output reg moving

  );

  // Encoding the states

    localparam IDLE      = 3'b000;
    localparam MOVE_UP    = 3'b001;
    localparam MOVE_DOWN  = 3'b010;
    localparam DOOR_OPEN  = 3'b011;
    localparam DOOR_CLOSE = 3'b100;

  // Declaring internal registers
    reg [2:0] current_state, next_state;
    reg [1:0] target_floor;

  // State Register
   always @(posedge clk or posedge reset)
   begin
   if (reset)
      current_state <= IDLE;
   else
      current_state <= next_state;
   end

  // Next-State Logic
   always @(*)
     begin
       case(current_state)

      IDLE:
  begin
  if(req_floor1)
```

```
    begin
       if(current_floor > 2'b01)
          next_state = MOVE_DOWN;
       else
          next_state = DOOR_OPEN;
    end

    else if(req_floor2)
    begin
       if(current_floor < 2'b10)
          next_state = MOVE_UP;
       else if(current_floor > 2'b10)
          next_state = MOVE_DOWN;
       else
          next_state = DOOR_OPEN;
    end

    else if(req_floor3)
    begin
       if(current_floor < 2'b11)
          next_state = MOVE_UP;
       else
          next_state = DOOR_OPEN;
    end

    else
       next_state = IDLE;
   end
       MOVE_UP:
   begin
    if(current_floor < target_floor)
       next_state = MOVE_UP;
    else
       next_state = DOOR_OPEN;
    end

     MOVE_DOWN:
   begin
    if(current_floor > target_floor)
       next_state = MOVE_DOWN;
    else
       next_state = DOOR_OPEN;
   end

       DOOR_OPEN:
       begin
          if(door_close)
```

```
            next_state = DOOR_CLOSE;
        else
            next_state = DOOR_OPEN;
    end

    DOOR_CLOSE:
    begin
        next_state = IDLE;
    end

    default:
    begin
        next_state = IDLE;
    end

 endcase
end

 // Current Floor Update Logic


always @(posedge clk or posedge reset)
begin
 if(reset)
 begin
    current_floor <= 2'b01;      // Start at Floor 1
 end

 else
 begin
    case(current_state)

        MOVE_UP:
        begin
            if(current_floor < target_floor)
                current_floor <= current_floor + 1;
            else
                current_floor <= current_floor;
        end

        MOVE_DOWN:
        begin
            if(current_floor > target_floor)
                current_floor <= current_floor - 1;
            else
                current_floor <= current_floor;
        end
```

```
        default:
   begin
   // Stay at current floor
    end
      endcase
     end
   end

  // Request Capture Logic


always @(posedge clk or posedge reset)
begin
   if(reset)
   begin
      target_floor <= 2'b01;   // Default Floor 1
   end

   else
   begin
      if(req_floor1)
         target_floor <= 2'b01;

      else if(req_floor2)
         target_floor <= 2'b10;

      else if(req_floor3)
         target_floor <= 2'b11;

      else
         target_floor <= target_floor;
   end
end
     // Output Logic


  always @(*)
  begin
   // Default Outputs
   direction = 2'b00;      // No Movement
   moving    = 1'b0;
   door_open  = 1'b0;

   case(current_state)

      IDLE:
```

```
      begin
        direction = 2'b00;
        moving    = 1'b0;
        door_open = 1'b0;
      end

      MOVE_UP:
      begin
        direction = 2'b01;
        moving    = 1'b1;
        door_open = 1'b0;
      end

      MOVE_DOWN:
      begin
        direction = 2'b10;
        moving    = 1'b1;
        door_open = 1'b0;
      end

      DOOR_OPEN:
      begin
        direction = 2'b00;
        moving    = 1'b0;
        door_open = 1'b1;
      end

      DOOR_CLOSE:
      begin
        direction = 2'b00;
        moving    = 1'b0;
        door_open = 1'b0;
      end

      default:
      begin
        direction = 2'b00;
        moving    = 1'b0;
        door_open = 1'b0;
      end

   endcase
end
endmodule
```

Explanation

Logic Explanation of Elevator Controller Code

The elevator controller is implemented using a **Finite State Machine (FSM)** technique to control the movement of a three-floor elevator. The design consists of five states: **IDLE, MOVE_UP, MOVE_DOWN, DOOR_OPEN, and DOOR_CLOSE**.

1. State Register Logic

The current state of the elevator is stored in a state register. On every positive edge of the clock, the controller updates its state according to the calculated next state.

- When the reset signal is activated, the elevator is initialized to the **IDLE** state.
- During normal operation, the current state changes according to the next-state logic.

2. Next-State Logic

The next state of the elevator is decided based on the current state, floor requests, current floor position, and door status.

IDLE State:

- The elevator remains idle until a floor request is received.
- If a request is made:
    - The controller compares the requested floor with the current floor.
    - If the requested floor is above the current floor, the elevator changes to **MOVE_UP** state.
    - If the requested floor is below the current floor, it changes to **MOVE_DOWN** state.
    - If the elevator is already at the requested floor, it directly moves to the **DOOR_OPEN** state.

MOVE_UP State:

- The elevator continues moving upward while the current floor is less than the target floor.
- Once the target floor is reached, the controller changes to the **DOOR_OPEN** state.

MOVE_DOWN State:

- The elevator continues moving downward while the current floor is greater than the target floor.

- When the target floor is reached, the controller changes to the **DOOR_OPEN** state.

DOOR_OPEN State:

- The door remains open until the door_close signal is received.
- After receiving the close command, the controller moves to the **DOOR_CLOSE** state.

DOOR_CLOSE State:

- After closing the door, the elevator returns to the **IDLE** state and waits for the next request.

3. Floor Update Logic

The current floor of the elevator is updated according to its movement state.

- During **MOVE_UP**, the current floor value is increased by one until the target floor is reached.
- During **MOVE_DOWN**, the current floor value is decreased by one until the target floor is reached.
- In other states, the elevator remains at the same floor.

The elevator starts from **Floor 1** after reset.

4. Request Capture Logic

The requested floor is stored in the target_floor register.

- When a floor request is detected:
    - Floor 1 request stores 01.
    - Floor 2 request stores 10.
    - Floor 3 request stores 11.
- This stored value is used by the controller to decide the direction of movement.

5. Output Control Logic

The output signals are generated based on the current FSM state.

- **IDLE:** Elevator is stationary with doors closed.
- **MOVE_UP:** Elevator moves upward and indicates upward direction.
- **MOVE_DOWN:** Elevator moves downward and indicates downward direction.

- **DOOR_OPEN:** Elevator stops and opens the door.
- **DOOR_CLOSE:** Elevator closes the door and prepares to return to idle operation.

Overall Working

The controller continuously monitors floor requests and controls elevator movement using FSM transitions. It determines the required direction by comparing the current floor and target floor, moves the elevator step-by-step, opens the door when the destination is reached, and returns to the idle state after completing the operation. This ensures proper sequencing of elevator movement and door control.

Testbench

```
module tb_elevator_controller;

    // Input Signals

reg clk;
reg reset;
reg req_floor1;
reg req_floor2;
reg req_floor3;
reg door_close;

   // Output Signals

wire [1:0] current_floor;
wire [1:0] direction;
wire moving;
wire door_open;

 // DUT (Design Under Test) Instantiation


elevator_controller uut (
   .clk(clk),
   .reset(reset),
   .req_floor1(req_floor1),
   .req_floor2(req_floor2),
   .req_floor3(req_floor3),
   .door_close(door_close),
   .current_floor(current_floor),
   .direction(direction),
   .moving(moving),
```

```
    .door_open(door_open)
);

    initial
    begin
    clk = 0;
    end

always
    #5 clk = ~clk;

    // Test Stimulus
initial
begin
    // Initialize Inputs
    reset = 1;
    req_floor1 = 0;
    req_floor2 = 0;
    req_floor3 = 0;
    door_close = 0;

    // Hold reset for 20 ns
    #20;
    reset = 0;


    // Test Case 1 : Floor 1 -> Floor 3
    // Hold request for one clock cycle

    $display("Test Case 1 : Move from Floor 1 to Floor 3");

    #10;
    req_floor3 = 1;

    #10;
    req_floor3 = 0;

    // Wait until elevator reaches Floor 3
    #100;

    // Passenger closes the door
    door_close = 1;

    #10;
    door_close = 0;
```

```
    // Test Case 2 : Floor 3 -> Floor 1
    // Hold request for one clock cycle


    #30;
    $display("Test Case 2 : Move from Floor 3 to Floor 1");
    req_floor1 = 1;

    #10;
    req_floor1 = 0;

    // Wait until elevator reaches Floor 1
    #100;

    door_close = 1;

    #10;
    door_close = 0;


    // Test Case 3 : Floor 1 -> Floor 2
    // Hold request for one clock cycle

    #30;
     $display("Test Case 3 : Move from Floor 1 to Floor 2");
    req_floor2 = 1;

    #10;
    req_floor2 = 0;

    // Wait until elevator reaches Floor 2
    #80;

    door_close = 1;

    #10;
    door_close = 0;


     // End Simulation
   #20;
   $display("Simulation Completed Successfully");
   $finish;
end


endmodule
```

Test Bench Explanation

The test bench is used to verify the functionality of the **elevator controller module** by applying different input conditions and observing the generated outputs.

- The test bench generates a **clock signal** with a time period of 10 ns using an always block.
- Initially, all input signals are set to their default values and the elevator controller is kept in the reset state for 20 ns.
- After releasing the reset, different floor requests are applied to check the elevator operation.

Test Cases Performed:

1. **Floor 1 to Floor 3 Movement**
    - A request for Floor 3 is applied.
    - The elevator moves upward until it reaches Floor 3.
    - After reaching the destination, the door close signal is applied.
2. **Floor 3 to Floor 1 Movement**
    - A request for Floor 1 is generated.
    - The elevator moves downward until it reaches Floor 1.
    - The door operation is tested after reaching the floor.
3. **Floor 1 to Floor 2 Movement**
    - A request for Floor 2 is applied.
    - The elevator moves upward by one floor and stops at Floor 2.
    - Door closing operation is verified.

The $display statements are used to show the progress of each test case during simulation. After all test cases are completed, the simulation is terminated using $finish.

This test bench verifies the correct operation of **FSM state transitions, floor movement, direction control, and door control logic** of the elevator controller.

Simulation Results

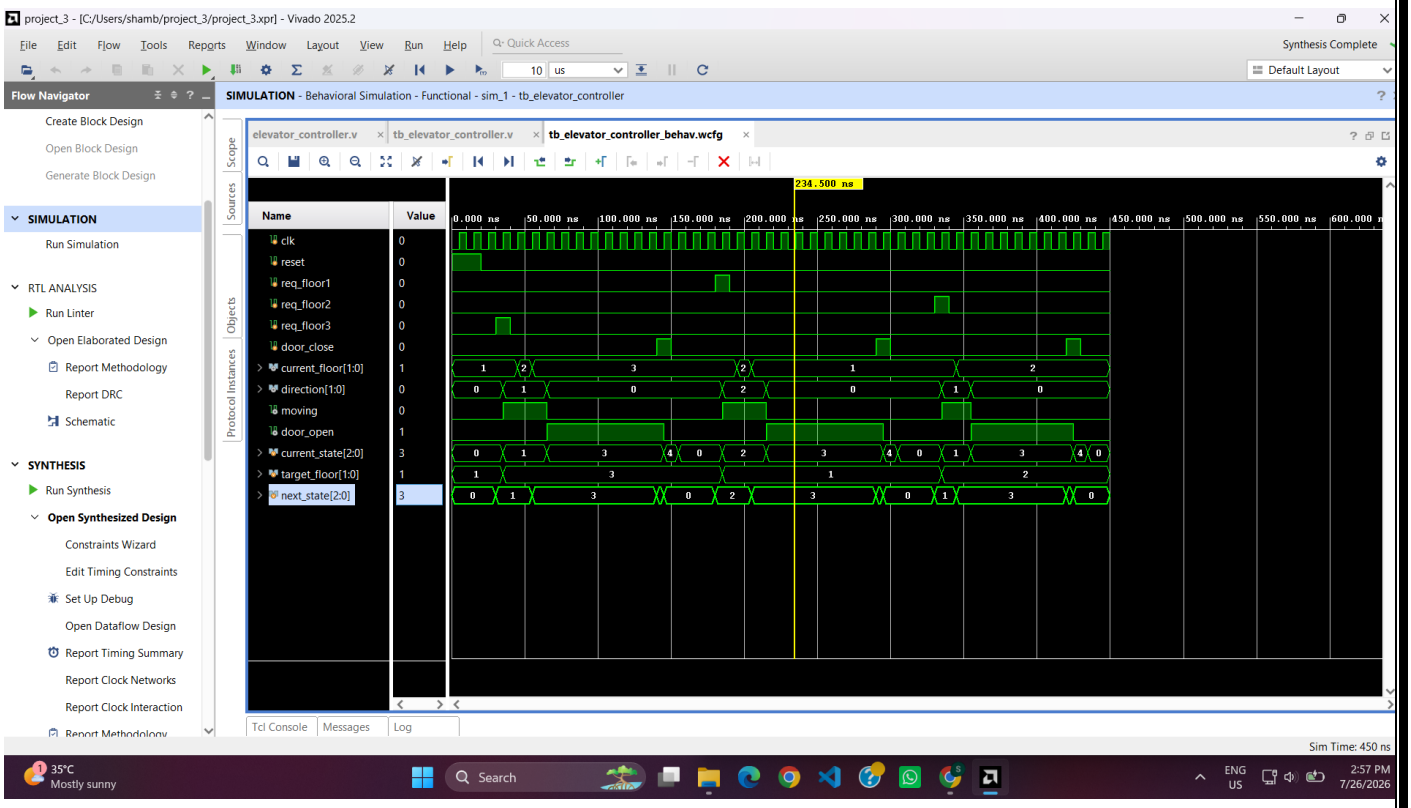


Fig 1.1 Simulation waveform (part1)

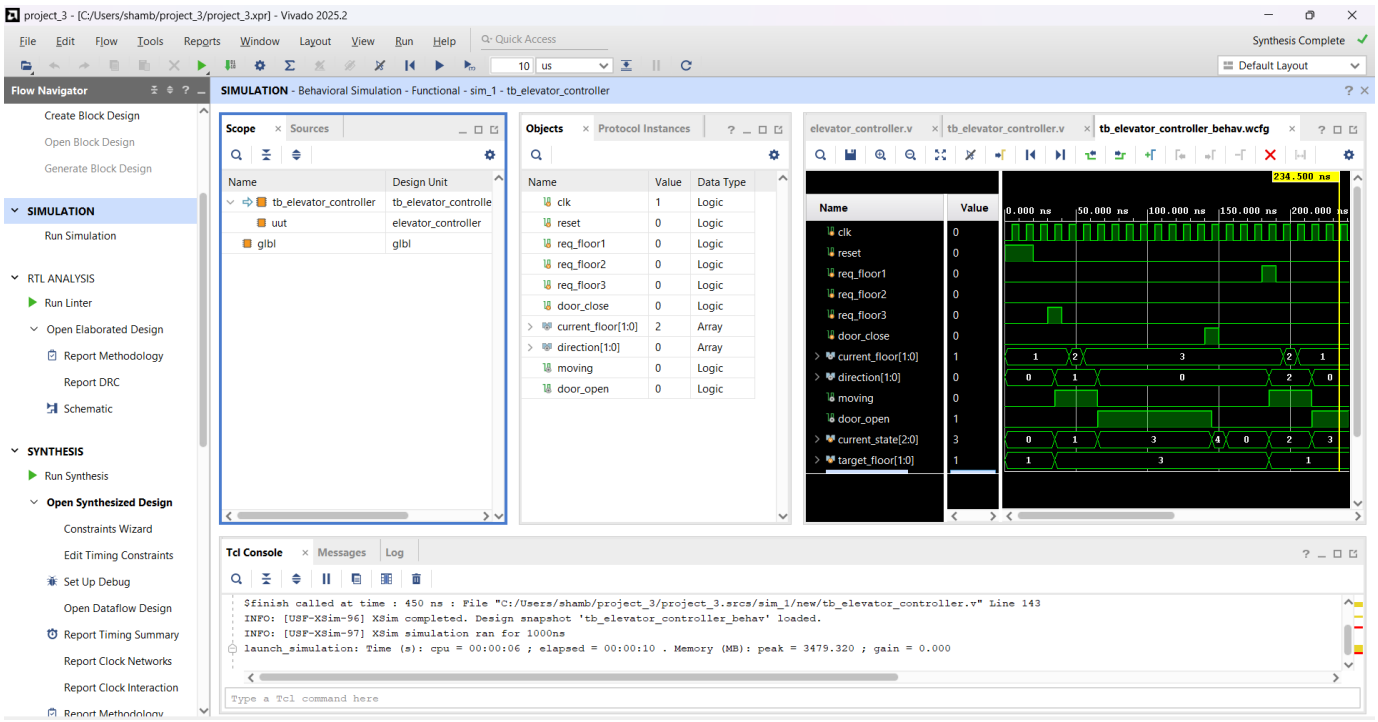


Fig 1.2 Simulation waveform (part 2)

RTL Schematic

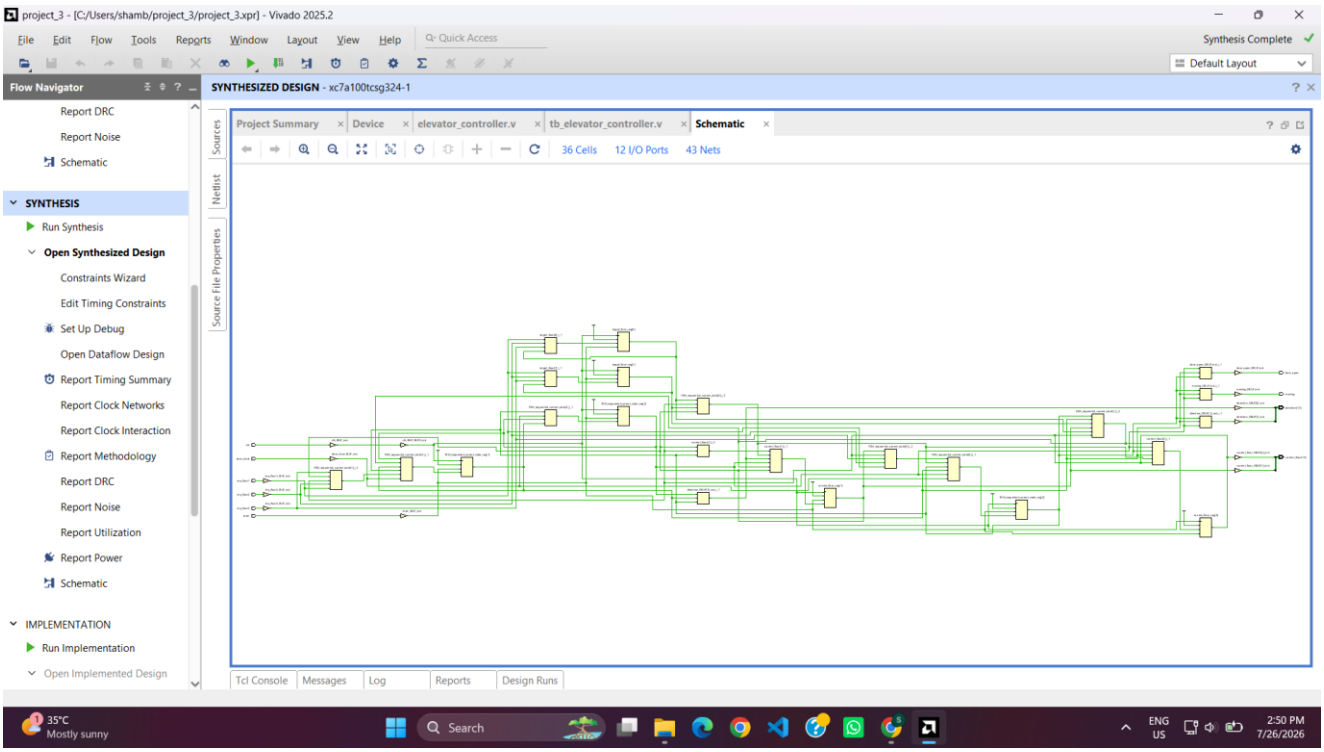


Fig 1.3 Full synthesized schematic

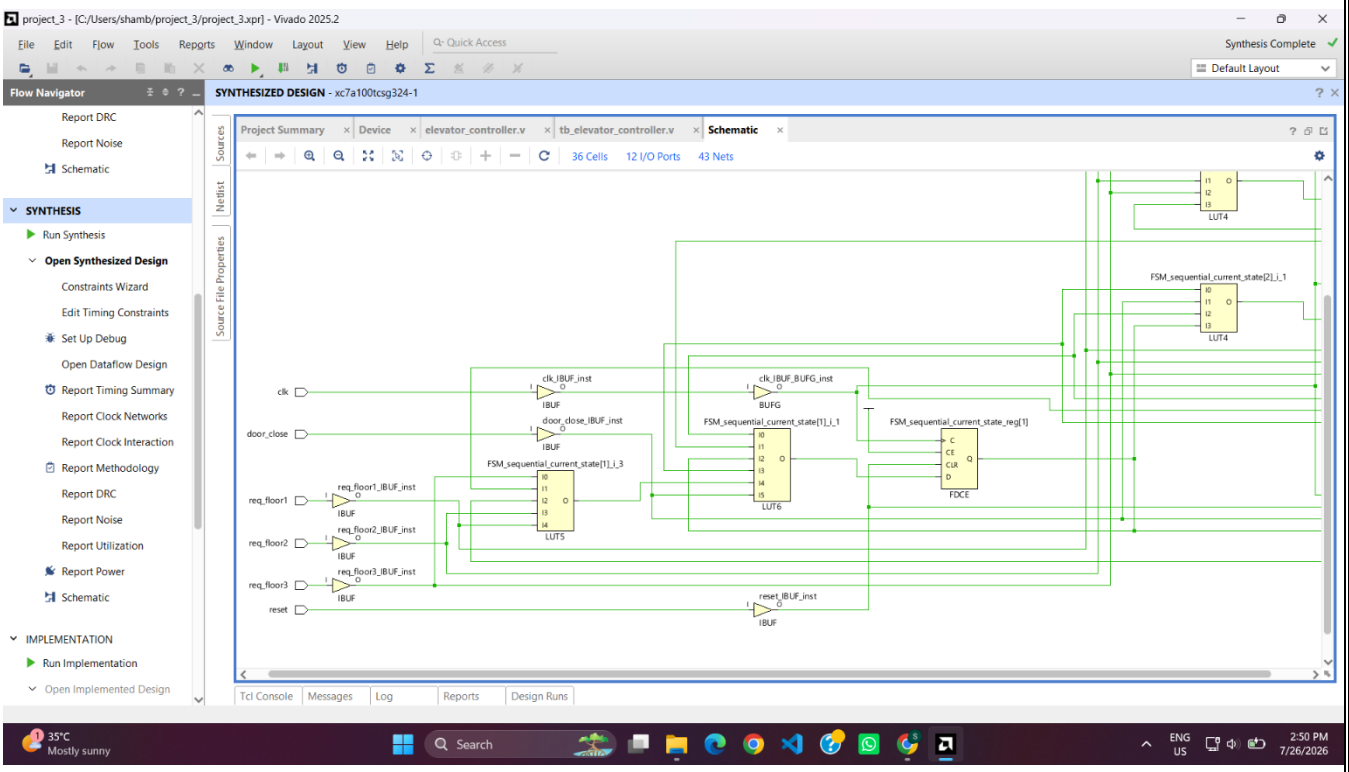


Fig 1.4 Zoomed synthesized schematic (part 1)

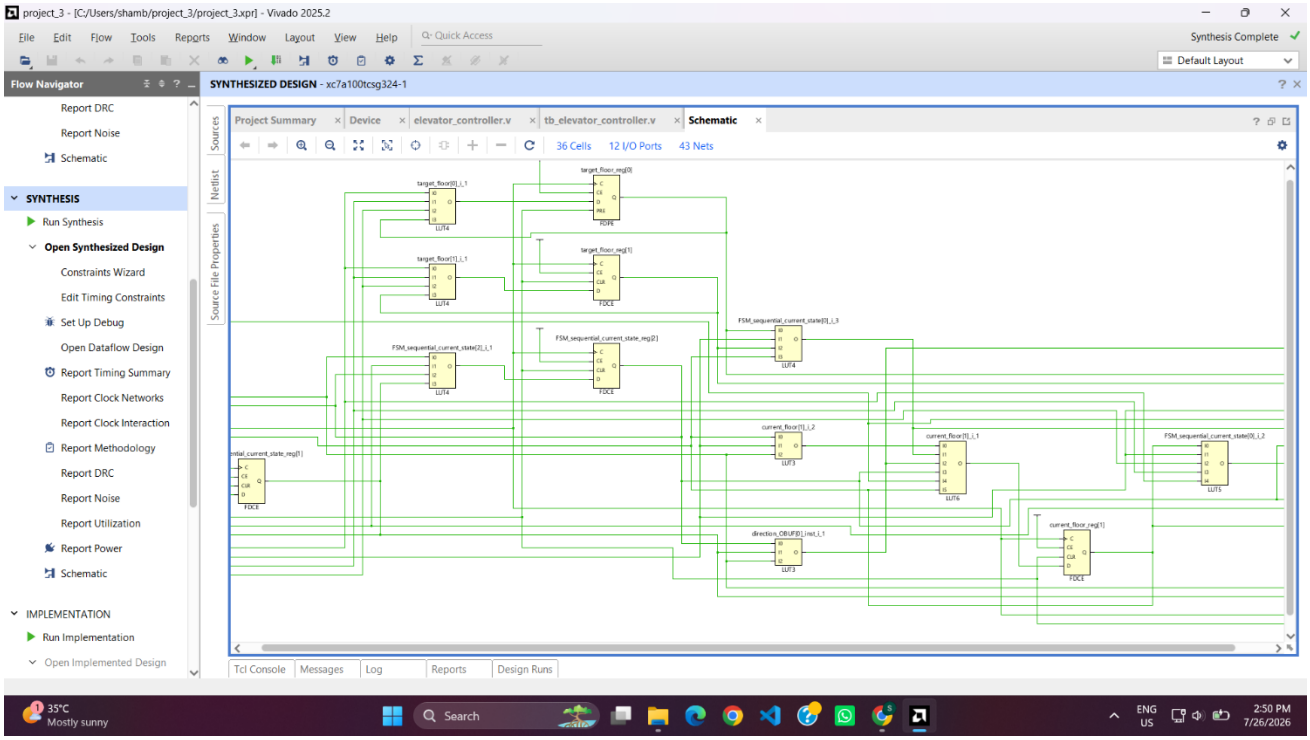


Fig 1.5 Zoomed synthesized schematic (part 2)

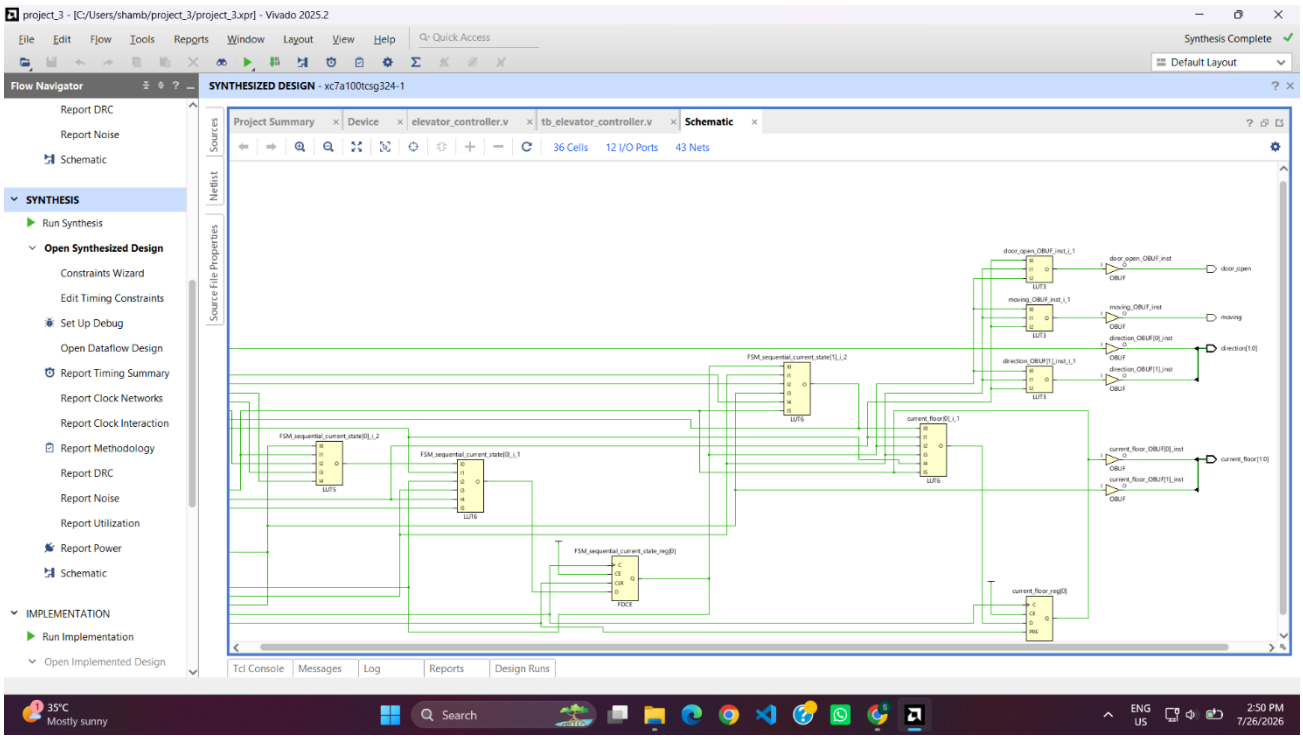
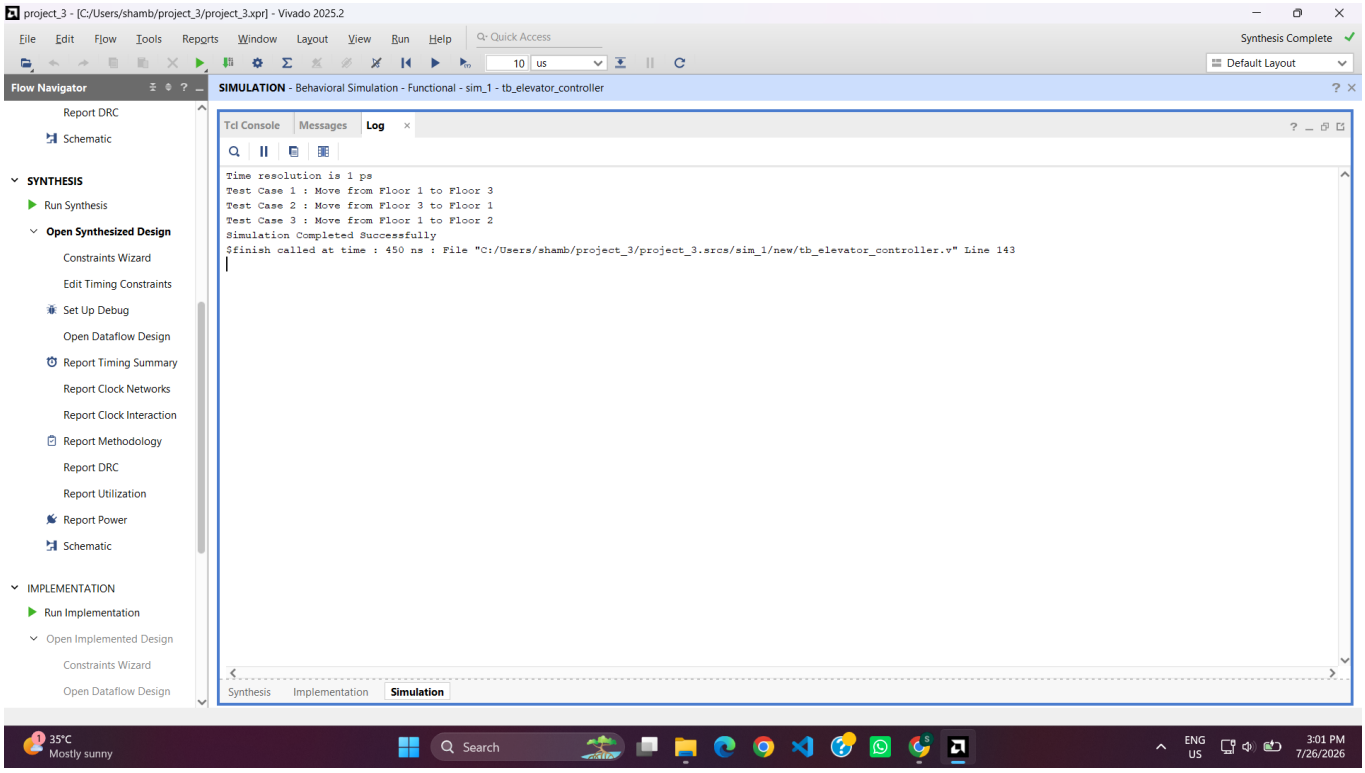


Fig 1.6 Zoomed synthesized schematic (part 3)

Simulation Log



Fing 1.7 Simulation Log

Functional Verification

Functional verification was performed to validate the correct operation of the **FSM-based Smart Elevator Controller**. The verification process focuses on checking FSM transitions, elevator movement, door control sequence, reset behavior, and safety conditions under different operating scenarios.

1. FSM States Used

The elevator controller is implemented using a **5-state Finite State Machine (FSM)**. Each state represents a specific operation of the elevator.

State	Description
IDLE	The elevator remains stationary and continuously monitors floor requests. No movement or door operation occurs in this state.
MOVE_UP	The elevator moves upward one floor at a time until the requested destination floor is reached.

State	Description
MOVE_DOWN	The elevator moves downward one floor at a time until the requested destination floor is reached.
DOOR_OPEN	The elevator stops at the requested floor and opens the door. The door remains open until a close command is received.
DOOR_CLOSE	The elevator closes the door and completes the operation before returning to the idle state.

The FSM uses **3-bit state encoding** because five different states are required.

State encoding used:

State	Binary Code
IDLE	000
MOVE_UP	001
MOVE_DOWN	010
DOOR_OPEN	011
DOOR_CLOSE	100

2. State Transition Logic

The transition between states is controlled by the current elevator position, requested floor, and door control signal.

IDLE → MOVE_UP / MOVE_DOWN / DOOR_OPEN

- Initially, the elevator remains in the IDLE state after reset.
- When a floor request is received, the controller compares the current floor with the requested floor.
- If the requested floor is above the current floor:
    - The FSM transitions to **MOVE_UP**.
- If the requested floor is below the current floor:
    - The FSM transitions to **MOVE_DOWN**.
- If the elevator is already at the requested floor:

- The FSM directly transitions to **DOOR_OPEN**.

MOVE_UP → MOVE_UP / DOOR_OPEN

- The elevator continues moving upward as long as:

$$Current\ Floor < Target\ Floor$$

- The current floor register is incremented by one after every clock cycle.
- When the target floor is reached, the FSM changes to **DOOR_OPEN**.

Example:

Floor 1 → Floor 3

Floor 1 → Floor 2 → Floor 3 → Door Open

MOVE_DOWN → MOVE_DOWN / DOOR_OPEN

- The elevator continues moving downward while:

$$Current\ Floor > Target\ Floor$$

- The current floor register is decremented by one after every clock cycle.
- When the destination floor is reached, the FSM transitions to **DOOR_OPEN**.

Example:

Floor 3 → Floor 1

Floor 3 → Floor 2 → Floor 1 → Door Open

DOOR_OPEN → DOOR_OPEN / DOOR_CLOSE

- After reaching the destination floor, the elevator stops and opens the door.
- The FSM remains in the DOOR_OPEN state until the door_close signal becomes high.
- Once the close command is received, the FSM moves to the DOOR_CLOSE state.

DOOR_CLOSE → IDLE

- After the door closing operation is completed, the FSM returns to the IDLE state.
- The controller is now ready to accept the next floor request.

3. Safety Conditions Implemented

The design incorporates several safety mechanisms to ensure reliable elevator operation.

1. Prevention of Invalid Movement

- The elevator changes direction only after comparing the current floor and requested floor.
- It cannot move upward when the destination floor is below the current floor.
- It cannot move downward when the destination floor is above the current floor.

This prevents incorrect movement commands.

2. Controlled Door Operation

- The door can open only when the elevator reaches the requested floor.
- The elevator remains stationary while the door is open.
- Movement and door opening cannot occur simultaneously.

This ensures passenger safety.

3. Reset Safety

- When reset is activated:
    - The FSM returns to the IDLE state.
    - The elevator position is initialized to Floor 1.
    - All operations are stopped.

This provides a known starting condition for the controller.

4. Valid FSM State Handling

- A default condition is included in the FSM logic.
- If an undefined state occurs due to any fault, the controller automatically returns to the IDLE state.

This prevents the controller from entering an unpredictable condition.

5. Target Floor Storage

- The requested floor is stored in the target_floor register.
- The elevator continues its operation based on the stored request even after the request signal is removed.

This ensures stable operation during movement.

4. Assumptions Made During Design

The following assumptions were considered while designing the elevator controller:

1. The elevator operates in a **three-floor building** with Floor 1, Floor 2, and Floor 3.
2. Only one floor request is processed at a time. Multiple simultaneous requests are not considered.
3. The elevator moves exactly **one floor per clock cycle**.
4. The clock signal represents the timing reference for elevator movement and FSM transitions.
5. The door opening and closing operation is assumed to complete within one FSM cycle.
6. The elevator starts from **Floor 1 after reset**.
7. The door_close signal is manually generated after passengers complete entry or exit.
8. Mechanical aspects such as motor control, obstacle detection, emergency stop, and overload protection are not included because this design focuses only on the digital controller logic.

Functional Verification Result

The test bench successfully verified the operation of the elevator controller by applying multiple floor requests:

- Floor 1 → Floor 3 movement
- Floor 3 → Floor 1 movement
- Floor 1 → Floor 2 movement

The simulation confirmed correct:

- FSM state transitions
- Floor position updates
- Upward and downward movement control
- Door opening and closing sequence

- Reset initialization

Therefore, the designed FSM-based elevator controller satisfies the required functional specifications and demonstrates reliable sequential control operation suitable for a digital VLSI design implementation.

Conclusion

The **Smart Elevator Controller using Verilog HDL** was successfully designed, simulated, and verified using a Finite State Machine (FSM) approach. The controller efficiently manages elevator movement, floor selection, and door operations for a three-floor building.

The FSM-based design successfully implemented different operating states such as idle, upward movement, downward movement, door opening, and door closing. Simulation results verified correct state transitions, floor updates, direction control, and safe door operation under different floor requests.

This project provided practical understanding of **RTL design, sequential logic, FSM modeling, and functional verification**, which are important concepts in VLSI digital system design. The developed controller can be further enhanced by adding features such as multiple request handling, emergency control, overload detection, and FPGA-based hardware implementation.